

TinyDB JDBC Driver

DBeaver (or any JDBC client)

|

java.sql.* API (DriverManager, Connection, Statement, ResultSet, DatabaseMetaData)

|

TinyDbDriver / TinyDbConnection / TinyDbStatement / TinyDbResultSet <-- THIS GUIDE

|

BasicQueryEngine / BasicPlanner / PersistentCatalog (companion Query Engine guide)

|

FileStorageEngine / TableHeap / WalManager (companion Storage Engine guide)

|

Disk

By the end of this guide you will have:

- A real **java.sql.Driver**, auto-discoverable by **DriverManager** via **META-INF/services**, accepting a **jdbc:tinydb:** connection URL that points at a local, embedded, file-based database — the same architecture SQLite's and H2's own embedded JDBC drivers use.
- A real **Connection**, **Statement**, **PreparedStatement**, and **ResultSet**, each backed directly by the already-built **BasicQueryEngine/BasicPlanner/Row** classes — no new query engine, no new storage engine, only the adapter layer between them and the JDBC SPI.
- A real, previously-missing **TransactionManager**, filling the exact gap the Query Engine guide named but never built, making **Connection.setAutoCommit(false)/commit()/rollback()** behave correctly against the Storage Engine's real WAL-backed transactions.
- A real **DatabaseMetaData** implementation covering exactly the methods DBeaver actually calls to populate its schema browser — **getCatalogs**, **getSchemas**, **getTables**, **getColumns**, **getPrimaryKeys** — built from the already-existing Catalog interface, not hand-waved.
- A concrete, literal walkthrough of loading the driver JAR into DBeaver and connecting to a real TinyDB data directory.

Project Structure

```
tinydb-server/
├── src/main/java/com/example/tinydb/server/
│   ├── protocol/
│   │   ├── Frame.java
│   │   ├── Request.java (sealed), Response.java (sealed)
│   │   └── ProtocolCodec.java
│   ├── auth/
│   │   └── CredentialStore.java, AuthHandshake.java
│   ├── TinyDbServer.java
│   └── ServerConnectionHandler.java
├── src/test/java/com/example/tinydb/server/
│   ├── ProtocolCodecRoundTripTest.java
│   ├── TwoProcessIntegrationTest.java
│   └── ConcurrentClientLoadTest.java
└──
```



```
tinydb-jdbc/
├── src/main/java/com/example/tinydb/jdbc/
│   ├── StatementExecutor.java
│   ├── EmbeddedStatementExecutor.java
│   ├── NetworkStatementExecutor.java
│   ├── RemoteCatalogSnapshot.java
│   ├── TinyDbConnection.java
│   └── TinyDbDriver.java
└──
```

8. Implementing a Real TransactionContext

```
public final class RealTransactionContext {

    private static final AtomicLong NEXT_TRANSACTION_ID = new AtomicLong(1);

    private final long transactionId;
    private volatile boolean completed; // true once committed or rolled back -- guards against
double-completion

    private RealTransactionContext(long transactionId) { this.transactionId = transactionId; }

    static RealTransactionContext allocate() {
        return new RealTransactionContext(NEXT_TRANSACTION_ID.getAndIncrement());
    }

    public long transactionId() { return transactionId; }
    public boolean isCompleted() { return completed; }
    void markCompleted() { completed = true; }
}
```

9. Implementing TransactionManager

```
public final class TransactionManager {

    private final StorageEngine storageEngine; // companion Storage Engine guide, §45

    public TransactionManager(StorageEngine storageEngine) { this.storageEngine = storageEngine; }

    public RealTransactionContext begin() {
        RealTransactionContext tx = RealTransactionContext.allocate(); // §8
        try {
            storageEngine.beginTransaction(tx.transactionId());
        } catch (IOException e) {
            throw new TinyDbRuntimeException("Failed to begin transaction", e);
        }
        return tx;
    }

    public void commit(RealTransactionContext tx) {
        if (tx.isCompleted()) throw new IllegalStateException("Transaction " + tx.transactionId() + " already completed");
        try {
            storageEngine.commitTransaction(tx.transactionId());
        } catch (IOException e) {
            throw new TinyDbRuntimeException("Failed to commit transaction " + tx.transactionId(), e);
        } finally {
            tx.markCompleted();
        }
    }

    public void rollback(RealTransactionContext tx) {
        if (tx.isCompleted()) throw new IllegalStateException("Transaction " + tx.transactionId() + " already completed");
        try {
            storageEngine.abortTransaction(tx.transactionId());
        } catch (IOException e) {
            throw new TinyDbRuntimeException("Failed to abort transaction " + tx.transactionId(), e);
        } finally {
            tx.markCompleted();
        }
    }
}
```

This is the exact class **BasicQueryEngine**'s constructor (companion Query Engine guide, §26) was already written expecting — nothing about **BasicQueryEngine.doQuery/doUpdate** needs to change to accept it; the gap named in §7 was purely that nothing had implemented it yet. **isCompleted()**'s guard against double-completion matters specifically for §17-§18: a JDBC **Connection.close()** that runs after an explicit **commit()** already happened must never attempt a second commit or an implicit rollback against an already-finished transaction.

10. The JDBC SPI Surface: Which Interfaces We Actually Need to Implement

Interface	Why it's needed	
java.sql.Driver	The entry point DriverManager (and DBeaver) discovers and calls to obtain a Connection	
java.sql.Connection	Owns the transaction boundary, and factories Statement/PreparedStatement/DatabaseMetaData	
java.sql.Statement	Executes a fixed SQL string, once, with no parameters	
java.sql.PreparedStatement	Executes a parameterized SQL string, safely, potentially many times	
java.sql.ResultSet	Iterates rows and columns of a query result	
java.sql.ResultSetMetaData	Describes a result set's columns — names, types — without materializing rows	
java.sql.DatabaseMetaData	Describes the database itself — catalogs, schemas, tables, columns — what DBeaver's schema browser is built entirely on	

Every one of these is a genuinely large interface in the real JDBC specification (ResultSet alone declares roughly 200 methods).

The Connection URL Format

```
jdbc:tinydb:/absolute/path/to/data/directory
jdbc:tinydb:./relative/path/to/data
```

Extracting a StatementExecutor Seam

```
public interface StatementExecutor {
    QueryResult execute(String sql, boolean isQuery) throws SQLException;

    boolean getAutoCommit();
    void setAutoCommit(boolean autoCommit) throws SQLException;
    void commit() throws SQLException;
    void rollback() throws SQLException;
    void close() throws SQLException;

    Catalog catalog(); // companion Query Engine guide, §33 -- what DatabaseMetaData (§26)
                        // is built on
}
```

TinyDbConnection to Delegate to a StatementExecutor

```
public final class TinyDbConnection implements Connection {
    private final StatementExecutor executor; // EmbeddedStatementExecutor OR
    NetworkStatementExecutor
    private boolean closed;

    public TinyDbConnection(StatementExecutor executor) { this.executor = executor; }

    @Override
    public Statement createStatement() throws SQLException {
        checkOpen();
        return new TinyDbStatement(this); // companion guide, §19 -- completely unchanged
    }

    @Override
    public PreparedStatement prepareStatement(String sql) throws SQLException {
        checkOpen();
        return new TinyDbPreparedStatement(this, sql); // companion guide, §21 -- completely unchanged
    }

    @Override
    public DatabaseMetaData getMetaData() throws SQLException {
        checkOpen();
        return new TinyDbDatabaseMetaData(executor.catalog()); // companion guide, §28 -- completely
        unchanged CLASS,
        // fed by whichever Catalog the executor exposes
    }

    QueryResult executeInternal(String sql, boolean isQuery) throws SQLException {
        checkOpen();
        return executor.execute(sql, isQuery); // the ENTIRE original branching logic now lives inside the
        executor, §9
    }

    @Override public boolean getAutoCommit() throws SQLException { checkOpen(); return
    executor.getAutoCommit(); }
    @Override public void setAutoCommit(boolean autoCommit) throws SQLException { checkOpen();
    executor.setAutoCommit(autoCommit); }
    @Override public void commit() throws SQLException { checkOpen(); executor.commit(); }
    @Override public void rollback() throws SQLException { checkOpen(); executor.rollback(); }

    @Override
    public void close() throws SQLException {
        if (closed) return;
        executor.close();
        closed = true;
    }

    private void checkOpen() throws SQLException { if (closed) throw new SQLException("Connection is
    closed"); }
}
```

Implementing EmbeddedStatementExecutor

```
public final class EmbeddedStatementExecutor implements StatementExecutor {
    private final TinyDbInstance instance; // companion guide, §13
    private boolean autoCommit = true;
    private RealTransactionContext currentTransaction;

    public EmbeddedStatementExecutor(TinyDbInstance instance) { this.instance = instance; }

    @Override public QueryResult execute(String sql, boolean isQuery) throws SQLException {
        try {
            if (autoCommit) {
                return isQuery ? instance.queryEngine().doQuery(sql) : instance.queryEngine().doUpdate(sql);
            }
            return executeWithinOpenTransaction(sql, isQuery);
        } catch (RuntimeException e) {
            throw new SQLException("Statement execution failed: " + e.getMessage(), e);
        }
    }

    private QueryResult executeWithinOpenTransaction(String sql, boolean isQuery) throws SQLException {
        if (currentTransaction == null) {
            currentTransaction = instance.transactionManager().begin();
        }
        BasicPlanner planner = instance.planner();
        if (isQuery) {
            Plan plan = planner.createQueryPlan(sql, currentTransaction);
            return materializeAsQueryResult(plan); // identical helper, companion guide §17
        }
        int affected = planner.executeUpdate(sql, currentTransaction);
        return QueryResult.ofAffected(affected);
    }

    @Override public boolean getAutoCommit() { return autoCommit; }

    @Override public void setAutoCommit(boolean autoCommit) throws SQLException {
        if (this.autoCommit == autoCommit) return;
        if (autoCommit && currentTransaction != null) {
            instance.transactionManager().commit(currentTransaction);
            currentTransaction = null;
        }
        this.autoCommit = autoCommit;
    }

    @Override public void commit() throws SQLException {
        if (autoCommit) throw new SQLException("commit() is not valid while autoCommit is true");
        if (currentTransaction != null) { instance.transactionManager().commit(currentTransaction); currentTransaction
= null; }
    }

    @Override public void rollback() throws SQLException {
        if (autoCommit) throw new SQLException("rollback() is not valid while autoCommit is true");
        if (currentTransaction != null) { instance.transactionManager().rollback(currentTransaction);
currentTransaction = null; }
    }

    @Override public void close() throws SQLException {
        if (!autoCommit && currentTransaction != null && !currentTransaction.isCompleted()) {
            instance.transactionManager().rollback(currentTransaction);
        }
    }

    @Override public Catalog catalog() { return instance.catalog(); }
}
```

The Wire Protocol: Framing and Message Types

```
public record Frame(byte[] payload) {

    public void writeTo(OutputStream out) throws IOException {
        DataOutputStream dataOut = new DataOutputStream(out);
        dataOut.writeInt(payload.length);
        dataOut.write(payload);
        dataOut.flush();
    }

    public static Frame readFrom(InputStream in) throws IOException {
        DataInputStream dataIn = new DataInputStream(in);
        int length = dataIn.readInt();
        if (length < 0 || length > MAX_FRAME_BYTES) throw new IOException("Invalid frame length: " + length); //§28
        byte[] payload = new byte[length];
        dataIn.readFully(payload); // blocks until exactly `length` bytes arrive, or throws EOFException on early close
        return new Frame(payload);
    }

    private static final int MAX_FRAME_BYTES = 64 * 1024 * 1024; // 64 MB -- a sanity bound, §28 explains why this matters
}
```

The Request Types

```
public sealed interface Request permits AuthRequest, ExecuteRequest, CommitRequest, RollbackRequest,
    SetAutoCommitRequest, FetchCatalogSnapshotRequest, CloseRequest {
}

public record AuthRequest(String username, String password) implements Request { } // §15-§16
public record ExecuteRequest(String sql, boolean isQuery) implements Request { } // §7's execute(...)
public record CommitRequest() implements Request { }
public record RollbackRequest() implements Request { }
public record SetAutoCommitRequest(boolean autoCommit) implements Request { }
public record FetchCatalogSnapshotRequest() implements Request { } // §26
public record CloseRequest() implements Request { }
```

The Response Types

```
public sealed interface Response permits AuthResponse, QueryResultResponse, ErrorResponse, AckResponse,
    CatalogSnapshotResponse {
}

public record AuthResponse(boolean success, String failureReason /* nullable */) implements Response { }
public record QueryResultResponse(List<Map<String, Object>> rows, int affectedRows) implements Response { }
public record ErrorResponse(String message, String sqlState) implements Response { } // §28 there
public record AckResponse() implements Response { } // commit/rollback/setAutoCommit/close
public record CatalogSnapshotResponse(List<SerializedTableSchema> tables) implements Response { } // §26

public record SerializedTableSchema(String tableName, List<SerializedColumn> columns) { }
public record SerializedColumn(String name, String dataType, boolean nullable) { } // dataType as a NAME, decoded
via DataType.valueOf(...)
```

Implementing Request/Response Serialization

```
public final class ProtocolCodec {

    public byte[] encode(Request request) throws IOException {
        ByteArrayOutputStream out = new ByteArrayOutputStream();
        DataOutputStream dataOut = new DataOutputStream(out);
        switch (request) {
            case AuthRequest r -> { dataOut.writeByte(1); writeString(dataOut, r.username());
writeString(dataOut, r.password()); }
            case ExecuteRequest r -> { dataOut.writeByte(2); writeString(dataOut, r.sql());
dataOut.writeBoolean(r.isQuery()); }
            case CommitRequest r -> dataOut.writeByte(3);
            case RollbackRequest r -> dataOut.writeByte(4);
            case SetAutoCommitRequest r -> { dataOut.writeByte(5); dataOut.writeBoolean(r.autoCommit());
}
            case FetchCatalogSnapshotRequest r -> dataOut.writeByte(6);
            case CloseRequest r -> dataOut.writeByte(7);
        }
        return out.toByteArray();
    }

    public Request decodeRequest(byte[] payload) throws IOException {
        DataInputStream dataIn = new DataInputStream(new ByteArrayInputStream(payload));
        int typeTag = dataIn.readByte();
        return switch (typeTag) {
            case 1 -> new AuthRequest(readString(dataIn), readString(dataIn));
            case 2 -> new ExecuteRequest(readString(dataIn), dataIn.readBoolean());
            case 3 -> new CommitRequest();
            case 4 -> new RollbackRequest();
            case 5 -> new SetAutoCommitRequest(dataIn.readBoolean());
            case 6 -> new FetchCatalogSnapshotRequest();
            case 7 -> new CloseRequest();
            default -> throw new IOException("Unknown request type tag: " + typeTag);
        };
    }

    private void writeString(DataOutputStream out, String value) throws IOException {
        byte[] bytes = value.getBytes(StandardCharsets.UTF_8);
        out.writeInt(bytes.length);
        out.write(bytes);
    }

    private String readString(DataInputStream in) throws IOException {
        int length = in.readInt();
        byte[] bytes = new byte[length];
        in.readFully(bytes);
        return new String(bytes, StandardCharsets.UTF_8);
    }

    // encode(Response)/decodeResponse(byte[]) follow the identical one-byte-type-tag-then-fields shape
    --
    // elided here since the pattern is fully established above; QueryResultResponse's row encoding
    reuses
    // writeString for keys and a small type-tagged value encoder for each Object (Integer/Long/Boolean/
    // Double/String/Instant/null), mirroring §26's DataType-name-based encoding for SerializedColumn.
}
```

Implementing the Auth Handshake

```
public interface CredentialStore {
    boolean verify(String username, String password);
}

public final class InMemoryCredentialStore implements CredentialStore {

    private final Map<String, String> hashedPasswordsByUsername; // username -> a salted hash,
    NEVER the plaintext password

    public InMemoryCredentialStore(Map<String, String> hashedPasswordsByUsername) {
        this.hashedPasswordsByUsername = hashedPasswordsByUsername;
    }

    @Override
    public boolean verify(String username, String password) {
        String expectedHash = hashedPasswordsByUsername.get(username);
        if (expectedHash == null) return false; // unknown username -- same failure shape as a wrong
password, deliberately
        return expectedHash.equals(hash(password));
    }

    private String hash(String password) {
        // A real implementation uses a slow, salted password hash (bcrypt/Argon2/PBKDF2) from a
well-reviewed
        // library -- NEVER a fast general-purpose hash like plain SHA-256 for password storage
specifically.
        // Named honestly here rather than implemented, for the same reason §18 doesn't hand-roll TLS:
password
        // hashing is exactly the kind of security-critical code you use a reviewed library for, not write
yourself.
        throw new UnsupportedOperationException("Wire a real password-hashing library here, e.g.
bcrypt");
    }
}
```

```
/** Runs on the SERVER, as the first step of every new connection, before any Request other than
 * AuthRequest (§12) is accepted at all. */
public boolean handleAuthHandshake(Socket socket, CredentialStore credentialStore) throws
IOException {
    Frame frame = Frame.readFrom(socket.getInputStream());
    Request request = new ProtocolCodec().decodeRequest(frame.payload());

    if (!(request instanceof AuthRequest auth)) {
        sendError(socket, "First message on a connection must be AuthRequest");
        return false;
    }
    boolean authenticated = credentialStore.verify(auth.username(), auth.password());
    sendResponse(socket, new AuthResponse(authenticated, authenticated ? null : "Invalid credentials"));
    return authenticated; // §20's ServerConnectionHandler closes the socket immediately if this is false
}
```

Why TLS Belongs Here, and Why We Don't Hand-Roll It

Without transport encryption, §16's AuthRequest — carrying a real username and password — travels across the network in plaintext, readable by anything positioned to intercept the connection. The fix is not "design a new encryption scheme" — it's "wrap the existing Socket in one already provided by the JDK":

```
SSLContext sslContext = SSLContext.getInstance("TLSv1.3");
sslContext.init(keyManagers, trustManagers, null); // standard JDK key/trust-store setup,
SSLServerSocketFactory factory = sslContext.getServerSocketFactory();
ServerSocket serverSocket = factory.createServerSocket(port); // IDENTICAL usage from here on to a
plain ServerSocket (§19)
```

The TinyDB Server Process: Architecture

```
ServerSocket.accept() loop (§19)
|
one accepted Socket per client
|
submitted as a task to a bounded worker pool (§19)
|
ServerConnectionHandler (§20-§21)
- runs §16's auth handshake FIRST
- then a request/response loop, decoding via ProtocolCodec (§14)
- holds ITS OWN EmbeddedStatementExecutor (§9), reused SERVER-SIDE
|
every handler's EmbeddedStatementExecutor wraps
the SAME, single, shared TinyDbInstance (§13 in the JDBC guide)
```

Implementing the Accept Loop and a Bounded Worker Pool

```
public final class TinyDbServer {

    private final int port;
    private final TinyDbInstance instance;           // ONE shared instance -- companion guide, §13
    private final CredentialStore credentialStore;    // §16
    private final ExecutorService workerPool;        // bounded, like the ExecutorFramework core+max
    private volatile boolean running;

    public TinyDbServer(int port, TinyDbInstance instance, CredentialStore credentialStore, int
maxConcurrentClients) {
        this.port = port;
        this.instance = instance;
        this.credentialStore = credentialStore;
        this.workerPool = new ThreadPoolExecutor(
            maxConcurrentClients / 2, maxConcurrentClients, // core/max--§22 explains bound's purpose
            60, TimeUnit.SECONDS,
            new LinkedBlockingQueue<>(100),                // a bounded queue -- clients beyond capacity
wait, not spawn unbounded threads
            new ThreadPoolExecutor.AbortPolicy());        // a client beyond queue capacity is rejected
outright, §28
    }

    public void start() throws IOException {
        running = true;
        try (ServerSocket serverSocket = new ServerSocket(port)) { // or SSLServerSocketFactory, §17
            while (running) {
                Socket clientSocket = serverSocket.accept(); // blocks until a client connects
                workerPool.submit(new ServerConnectionHandler(clientSocket, instance, credentialStore)); //
§20
            }
        }
    }

    public void stop() { running = false; workerPool.shutdown(); }
}
```

Implementing ServerConnectionHandler

```
public final class ServerConnectionHandler implements Runnable {

    private final Socket socket;
    private final TinyDbInstance instance;
    private final CredentialStore credentialStore;
    private final ProtocolCodec codec = new ProtocolCodec(); // §14

    public ServerConnectionHandler(Socket socket, TinyDbInstance instance, CredentialStore
credentialStore) {
        this.socket = socket;
        this.instance = instance;
        this.credentialStore = credentialStore;
    }

    @Override
    public void run() {
        try (socket) {
            if (!authenticate()) return; // §15-§16 -- closes the socket immediately on failure, no further
messages read

            EmbeddedStatementExecutor executor = new EmbeddedStatementExecutor(instance); // §9,
ONE PER CONNECTION -- §18
            while (!socket.isClosed()) {
                Frame requestFrame = Frame.readFrom(socket.getInputStream());
                Request request = codec.decodeRequest(requestFrame.payload());
                Response response = dispatch(request, executor); // §21
                new Frame(codec.encode(response)).writeTo(socket.getOutputStream());
                if (request instanceof CloseRequest) break; // the client is done -- stop reading further frames
            }
        } catch (EOFException e) {
            // the client closed its socket without sending a CloseRequest -- treated identically to one that did,
            // after making sure any open transaction is rolled back (the try-with-resources below still runs)
        } catch (IOException e) {
            // a genuine network error mid-connection -- §28 discusses this case specifically
        } finally {
            // if authenticate() succeeded but the loop exited abnormally, any open transaction must still be
            // resolved -- NOT left dangling open against the shared TinyDbInstance indefinitely.
        }
    }

    private boolean authenticate() throws IOException { /* §16's handleAuthHandshake, inlined here */
        return true; }
}
```

Dispatching a Request to the Executor

```
private Response dispatch(Request request, EmbeddedStatementExecutor executor) {
    try {
        return switch (request) {
            case ExecuteRequest r -> {
                QueryResult result = executor.execute(r.sql(), r.isQuery()); // §7's seam, called SERVER-SIDE
                yield toQueryResultResponse(result);
            }
            case CommitRequest r -> { executor.commit(); yield new AckResponse(); }
            case RollbackRequest r -> { executor.rollback(); yield new AckResponse(); }
            case SetAutoCommitRequest r -> { executor.setAutoCommit(r.autoCommit()); yield new
AckResponse(); }
            case FetchCatalogSnapshotRequest r -> toCatalogSnapshotResponse(executor.catalog()); // §26
            case CloseRequest r -> { executor.close(); yield new AckResponse(); }
            case AuthRequest r -> new ErrorResponse("Already authenticated on this connection", "08001");
        };
    } catch (SQLException e) {
        return new ErrorResponse(e.getMessage(), e.getSQLState()); // §14 in the companion JDBC
guide's exception mapping
    }
}
```

Implementing NetworkStatementExecutor

```
public final class NetworkStatementExecutor implements StatementExecutor {

    private final Socket socket;
    private final ProtocolCodec codec = new ProtocolCodec(); // §14 -- IDENTICAL codec, used on both
ends of the wire
    private boolean autoCommit = true; // mirrored client-side purely so getAutoCommit() needs no round
trip

    public NetworkStatementExecutor(String host, int port, String username, String password) throws
SQLException {
        try {
            this.socket = new Socket(host, port); // or an SSLSocket, §17 -- identical usage from here on
            Response authResponse = send(new AuthRequest(username, password)); // §15-§16
            if (!(authResponse instanceof AuthResponse auth) || !auth.success()) {
                throw new SQLException("Authentication failed" + (authResponse instanceof AuthResponse a
? ": " + a.failureReason() : ""));
            }
        } catch (IOException e) {
            throw new SQLException("Could not connect to TinyDB server at " + host + ":" + port, e);
        }
    }

    @Override
    public QueryResult execute(String sql, boolean isQuery) throws SQLException {
        Response response = send(new ExecuteRequest(sql, isQuery)); // §12
        return switch (response) {
            case QueryResultResponse r -> toQueryResult(r); // rebuilds a List<Row> from r.rows(), the
SAME §24 needs
            case ErrorResponse e -> throw new SQLException(e.message(), e.sqlState());
            default -> throw new SQLException("Unexpected response type for ExecuteRequest: " +
```

```

response);
    };
}

@Override public boolean getAutoCommit() { return autoCommit; }

@Override
public void setAutoCommit(boolean autoCommit) throws SQLException {
    sendExpectingAck(new SetAutoCommitRequest(autoCommit)); // §12 -- the SERVER's executor
    (§9) holds the real state
    this.autoCommit = autoCommit; // mirrored, purely to answer getAutoCommit() without a round trip
}

@Override public void commit() throws SQLException { sendExpectingAck(new CommitRequest()); }
@Override public void rollback() throws SQLException { sendExpectingAck(new RollbackRequest()); }

@Override
public void close() throws SQLException {
    try {
        sendExpectingAck(new CloseRequest());
    } finally {
        try { socket.close(); } catch (IOException ignored) {}
    }
}

@Override
public Catalog catalog() {
    return new RemoteCatalogSnapshot(this); // §26 -- fetches lazily, on first actual use
}

/** Package-private -- called only by RemoteCatalogSnapshot (§26), never directly by JDBC-facing
code. */
CatalogSnapshotResponse fetchCatalogSnapshot() throws SQLException {
    Response response = send(new FetchCatalogSnapshotRequest()); // §12
    if (response instanceof CatalogSnapshotResponse snapshot) return snapshot;
    if (response instanceof ErrorResponse e) throw new SQLException(e.message(), e.sqlState());
    throw new SQLException("Unexpected response type for FetchCatalogSnapshotRequest: " +
response);
}

private Response send(Request request) throws SQLException {
    try {
        new Frame(codec.encode(request)).writeTo(socket.getOutputStream());
        Frame responseFrame = Frame.readFrom(socket.getInputStream());
        return codec.decodeResponse(responseFrame.payload());
    } catch (IOException e) {
        throw new SQLException("Network error communicating with TinyDB server", e); // §29
discusses this case
    }
}

private void sendExpectingAck(Request request) throws SQLException {
    Response response = send(request);
    if (response instanceof ErrorResponse e) throw new SQLException(e.message(), e.sqlState());
}
}

```

TinyDbDriver Dispatches by URL Scheme, Constructing the Right Executor

```
@Override
public Connection connect(String url, Properties info) throws SQLException {
    if (url.startsWith("jdbc:tinydb:file:") || (url.startsWith("jdbc:tinydb:") && !url.contains(":/"))) {
        return connectEmbedded(url); // companion guide's §15, unchanged -- constructs an
        EmbeddedStatementExecutor
    }
    if (url.startsWith("jdbc:tinydb://")) {
        return connectRemote(url, info); // NEW, this guide
    }
    return null; // §16 in the companion guide's contract -- an unrecognized URL returns null, never throws
}

private Connection connectRemote(String url, Properties info) throws SQLException {
    URI uri = URI.create(url.substring("jdbc:tinydb:".length())); // strips down to "//host:port/database"
    String host = uri.getHost();
    int port = uri.getPort();
    String username = info.getProperty("user");
    String password = info.getProperty("password");

    StatementExecutor executor = new NetworkStatementExecutor(host, port, username, password); //
    §23
    return new TinyDbConnection(executor); // §8 -- the SAME Connection class the embedded path also
    returns
}
```

TinyDbDriver Dispatches by URL Scheme, Constructing the Right Executor

```
public final class RemoteCatalogSnapshot implements Catalog {

    private final NetworkStatementExecutor executor;
    private List<TableSchema> cachedTables; // fetched lazily, on first metadata call -- §29 discusses
    staleness

    public RemoteCatalogSnapshot(NetworkStatementExecutor executor) { this.executor = executor; }

    @Override
    public List<TableSchema> listTables() {
        ensureFetched();
        return cachedTables;
    }

    @Override
    public TableSchema getTable(String tableName) {
        return listTables().stream().filter(t -> t.tableName().equals(tableName)).findFirst().orElse(null);
    }

    @Override public boolean tableExists(String tableName) { return getTable(tableName) != null; }
    @Override public void createTable(TableSchema schema) { throw new
    UnsupportedOperationException("Read-only snapshot"); }
```

```

@Override public void dropTable(String tableName) { throw new
UnsupportedOperationException("Read-only snapshot"); }

private void ensureFetched() {
    if (cachedTables != null) return;
    try {
        CatalogSnapshotResponse snapshot = executor.fetchCatalogSnapshot(); // sends
FetchCatalogSnapshotRequest, §12
        cachedTables = snapshot.tables().stream().map(this::toTableSchema).toList();
    } catch (SQLException e) {
        throw new RuntimeException("Failed to fetch catalog metadata from server", e);
    }
}

private TableSchema toTableSchema(SerializedTableSchema serialized) {
    List<Column> columns = serialized.columns().stream()
        .map(c -> new Column(c.name(), DataType.valueOf(c.dataType()), c.nullable()))
        .toList();
    return new TableSchema(serialized.tableName(), columns);
}
}

```

Server Configuration and Startup

```

public final class TinyDbServerMain {
    public static void main(String[] args) throws Exception {
        Path dataDirectory = Path.of(args[0]);
        int port = Integer.parseInt(args[1]);

        TinyDbInstance instance = TinyDbInstance.open(dataDirectory);
        CredentialStore credentialStore = loadCredentialStore(args[2]);
        TinyDbServer server = new TinyDbServer(port, instance, credentialStore, 200);
        Runtime.getRuntime().addShutdownHook(new Thread(server::stop));
        server.start(); // blocks -- the accept loop, §19
    }
}

```

Registering a shutdown hook that calls **server.stop()** matters for a reason specific to this guide's whole design: **stop()** only stops accepting new connections and shuts down the worker pool — it deliberately does not forcibly kill in-flight **ServerConnectionHandlers** mid-statement, giving already-connected clients a chance to finish or explicitly close cleanly rather than having their in-flight transaction torn out from under them by a process kill.

Follow-up:

What About a Very Large Result Set Over the Network?

Handling Connection Loss, Timeouts, and Partial Writes